

The Mathlib Contribution Process

Ruth Plümer

June 9, 2026

Index

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

The Review Process

Mathlib contributions happen via pull requests on GitHub, which need to be **reviewed** before they are adopted.

Why do we need reviewing?

Why do we need reviewing?

- **Catch malicious code**

Why do we need reviewing?

- **Catch malicious code**
- **Quality control**

Why do we need reviewing?

- **Catch malicious code**
- **Quality control**
 - ▶ Ensure that style guide is followed

Why do we need reviewing?

- **Catch malicious code**
- **Quality control**
 - ▶ Ensure that style guide is followed
 - ▶ Ensure scalability

Why do we need reviewing?

- **Catch malicious code**
- **Quality control**
 - ▶ Ensure that style guide is followed
 - ▶ Ensure scalability
 - ▶ Correct placement of new declarations

Why do we need reviewing?

- **Catch malicious code**
- **Quality control**
 - ▶ Ensure that style guide is followed
 - ▶ Ensure scalability
 - ▶ Correct placement of new declarations
 - ▶ Avoid duplications

Why do we need reviewing?

- **Catch malicious code**
- **Quality control**
 - ▶ Ensure that style guide is followed
 - ▶ Ensure scalability
 - ▶ Correct placement of new declarations
 - ▶ Avoid duplications
- **Encouragement of new contributors**

Why do we need reviewing?

- **Catch malicious code**
- **Quality control**
 - ▶ Ensure that style guide is followed
 - ▶ Ensure scalability
 - ▶ Correct placement of new declarations
 - ▶ Avoid duplications
- **Encouragement of new contributors**
- **Teaching opportunities**

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

What belongs into Mathlib?

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“
- Exact scope is still subject of debate

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“
- Exact scope is still subject of debate
- **Rule of thumb:**

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“
- Exact scope is still subject of debate
- **Rule of thumb:**
 - ▶ Contents of a typical undergraduate / graduate mathematics course or research-level mathematics group

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“
- Exact scope is still subject of debate
- **Rule of thumb:**
 - ▶ Contents of a typical undergraduate / graduate mathematics course or research-level mathematics group
 - ▶ Should be contained in the maintainers' research interests

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“
- Exact scope is still subject of debate
- **Rule of thumb:**
 - ▶ Contents of a typical undergraduate / graduate mathematics course or research-level mathematics group
 - ▶ Should be contained in the maintainers' research interests
 - ▶ If in doubt, ask in the `#mathlib`-Channel on Zulip

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“
- Exact scope is still subject of debate
- **Rule of thumb:**
 - ▶ Contents of a typical undergraduate / graduate mathematics course or research-level mathematics group
 - ▶ Should be contained in the maintainers' research interests
 - ▶ If in doubt, ask in the `#mathlib`-Channel on Zulip
- Possibly better places for your idea:

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“
- Exact scope is still subject of debate
- **Rule of thumb:**
 - ▶ Contents of a typical undergraduate / graduate mathematics course or research-level mathematics group
 - ▶ Should be contained in the maintainers' research interests
 - ▶ If in doubt, ask in the `#mathlib`-Channel on Zulip
- Possibly better places for your idea:
 - ▶ CSLib

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“
- Exact scope is still subject of debate
- **Rule of thumb:**
 - ▶ Contents of a typical undergraduate / graduate mathematics course or research-level mathematics group
 - ▶ Should be contained in the maintainers' research interests
 - ▶ If in doubt, ask in the `#mathlib`-Channel on Zulip
- Possibly better places for your idea:
 - ▶ CSLib
 - ▶ Physlib

What belongs into Mathlib?

- **Not:** „all of Mathlib and related areas“
- Exact scope is still subject of debate
- **Rule of thumb:**
 - ▶ Contents of a typical undergraduate / graduate mathematics course or research-level mathematics group
 - ▶ Should be contained in the maintainers' research interests
 - ▶ If in doubt, ask in the `#mathlib`-Channel on Zulip
- Possibly better places for your idea:
 - ▶ CSLib
 - ▶ Physlib
 - ▶ Standalone projects depending on Mathlib, notably the combinatorial game theory repository

Where to start?

Almost always welcome:

Where to start?

Almost always welcome:

- Add a single lemma to a theory that's already in Mathlib

Where to start?

Almost always welcome:

- Add a single lemma to a theory that's already in Mathlib
- Extend a theory that's already in Mathlib

Where to start?

Almost always welcome:

- Add a single lemma to a theory that's already in Mathlib
- Extend a theory that's already in Mathlib
- Fix typos

Where to start?

Almost always welcome:

- Add a single lemma to a theory that's already in Mathlib
- Extend a theory that's already in Mathlib
- Fix typos
- Work on a project from the **issues list**

Where to start?

Almost always welcome:

- Add a single lemma to a theory that's already in Mathlib
- Extend a theory that's already in Mathlib
- Fix typos
- Work on a project from the **issues list**
 - ▶ You can filter for **good first issues** if you're new!

Where to start?

Almost always welcome:

- Add a single lemma to a theory that's already in Mathlib
- Extend a theory that's already in Mathlib
- Fix typos
- Work on a project from the **issues list**
 - ▶ You can filter for **good first issues** if you're new!
- Add a new theory *within the scope of Mathlib*

Where to start?

Almost always welcome:

- Add a single lemma to a theory that's already in Mathlib
- Extend a theory that's already in Mathlib
- Fix typos
- Work on a project from the **issues list**
 - ▶ You can filter for **good first issues** if you're new!
- Add a new theory *within the scope of Mathlib*
 - ▶ Check the **list of undergraduate topics missing in Mathlib!**

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Mathlib's values

Mathlib's values

- Trust & Reliability

Mathlib's values

- Trust & Reliability
- Open source

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions
- Weak hypotheses - strong conclusions

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions
- Weak hypotheses - strong conclusions
- Classical logic preferred over constructive logic

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions
- Weak hypotheses - strong conclusions
- Classical logic preferred over constructive logic
 - ▶ Most of modern mathematics research depends on the law of excluded middle

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions
- Weak hypotheses - strong conclusions
- Classical logic preferred over constructive logic
 - ▶ Most of modern mathematics research depends on the law of excluded middle
 - ▶ Many widely-accepted results are not true without it

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions
- Weak hypotheses - strong conclusions
- Classical logic preferred over constructive logic
 - ▶ Most of modern mathematics research depends on the law of excluded middle
 - ▶ Many widely-accepted results are not true without it
- Downstream projects should face as few problems as possible

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions
- Weak hypotheses - strong conclusions
- Classical logic preferred over constructive logic
 - ▶ Most of modern mathematics research depends on the law of excluded middle
 - ▶ Many widely-accepted results are not true without it
- Downstream projects should face as few problems as possible
- Accessibility to non-formalists / non-programmers

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions
- Weak hypotheses - strong conclusions
- Classical logic preferred over constructive logic
 - ▶ Most of modern mathematics research depends on the law of excluded middle
 - ▶ Many widely-accepted results are not true without it
- Downstream projects should face as few problems as possible
- Accessibility to non-formalists / non-programmers
 - ▶ Create informative docstrings

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions
- Weak hypotheses - strong conclusions
- Classical logic preferred over constructive logic
 - ▶ Most of modern mathematics research depends on the law of excluded middle
 - ▶ Many widely-accepted results are not true without it
- Downstream projects should face as few problems as possible
- Accessibility to non-formalists / non-programmers
 - ▶ Create informative docstrings
 - ▶ Creating tools (like NNG) that help with accessibility is greatly appreciated!

Mathlib's values

- Trust & Reliability
- Open source
- Maintainability
 - ▶ Keep in mind: *Zero sorrys is a starting point, not an end point!*
- General definitions
- Weak hypotheses - strong conclusions
- Classical logic preferred over constructive logic
 - ▶ Most of modern mathematics research depends on the law of excluded middle
 - ▶ Many widely-accepted results are not true without it
- Downstream projects should face as few problems as possible
- Accessibility to non-formalists / non-programmers
 - ▶ Create informative docstrings
 - ▶ Creating tools (like NNG) that help with accessibility is greatly appreciated!

Additional *Code of Conduct*

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

A few words on style

Mathlib has a **style guide**, make sure your additions adhere to it!

A few words on style

Mathlib has a **style guide**, make sure your additions adhere to it!

Relatively strict **naming conventions** for theorems, lemmas and definitions, make sure to read them!

A few words on style

Mathlib has a **style guide**, make sure your additions adhere to it!

Relatively strict **naming conventions** for theorems, lemmas and definitions, make sure to read them!

→ They help other users tremendously to find the theorem they are looking for!

A few words on style

Mathlib has a **style guide**, make sure your additions adhere to it!

Relatively strict **naming conventions** for theorems, lemmas and definitions, make sure to read them!

→ They help other users tremendously to find the theorem they are looking for!

Style-only contributions are welcome if they fix violations of the style guide.

A few words on style

Mathlib has a **style guide**, make sure your additions adhere to it!

Relatively strict **naming conventions** for theorems, lemmas and definitions, make sure to read them!

→ They help other users tremendously to find the theorem they are looking for!

Style-only contributions are welcome if they fix violations of the style guide.

→ Discuss other style-only contributions on Zulip beforehand!

A few words on style

Which stylistic problems can you spot here?

```
1  -- do NOT write code in this style!
2  theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α)
3  : a*b*c*d=a*(b*(c*d)) :=
4  by
5  rw [mul_assoc,
6  mul_assoc]
7
```

A few words on style

Which stylistic problems can you spot here?

```
1  -- do NOT write code in this style!
2  theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α)
3  : a*b*c*d=a*(b*(c*d)) :=
4  by
5  rw [mul_assoc,
6  mul_assoc]
7
```

- Lines should not start with :

A few words on style

Which stylistic problems can you spot here?

```
1  -- do NOT write code in this style!
2  theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α)
3  : a*b*c*d=a*(b*(c*d)) :=
4  by
5  rw [mul_assoc,
6  mul_assoc]
7
```

- Lines should not start with :
- Spaces around the binary operation `*` are missing

A few words on style

Which stylistic problems can you spot here?

```
1  -- do NOT write code in this style!
2  theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α)
3  : a*b*c*d=a*(b*(c*d)) :=
4  by
5  rw [mul_assoc,
6  mul_assoc]
7
```

- Lines should not start with :
- Spaces around the binary operation `*` are missing
- `by` should never be in its own line

A few words on style

Which stylistic problems can you spot here?

```
1  -- do NOT write code in this style!
2  theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α)
3  : a*b*c*d=a*(b*(c*d)) :=
4  by
5  rw [mul_assoc,
6  mul_assoc]
7
```

- Lines should not start with :
- Spaces around the binary operation `*` are missing
- `by` should never be in its own line
- Tactic blocks should be indented

A few words on style

Which stylistic problems can you spot here?

```
1  -- do NOT write code in this style!
2  theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α)
3  : a*b*c*d=a*(b*(c*d)) :=
4  by
5  rw [mul_assoc,
6  mul_assoc]
7
```

- Lines should not start with :
- Spaces around the binary operation `*` are missing
- `by` should never be in its own line
- Tactic blocks should be indented
- The `rw`-block is unnecessarily split into two lines

A few words on style

```
1  -- Good version of the previous theorem
2  theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α) :
3      a * b * c * d = a * (b * (c * d)) := by
4      rw [mul_assoc, mul_assoc]
```

A few words on style

```
1  -- Good version of the previous theorem
2  theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α) :
3      a * b * c * d = a * (b * (c * d)) := by
4      rw [mul_assoc, mul_assoc]
5
```

Many stylistic problems will already be caught by **style linters** (if you don't disable them!)

Some remarks on naming conventions

Capitalization rules:

Some remarks on naming conventions

Capitalization rules:

- 1 theorems, lemmas and names of proofs are in `snake_case`.

Some remarks on naming conventions

Capitalization rules:

- ① theorems, lemmas and names of proofs are in `snake_case`.
- ② structures, classes and inductive types are in `UpperCamelCase`

Some remarks on naming conventions

Capitalization rules:

- ① theorems, lemmas and names of proofs are in `snake_case`.
- ② structures, classes and inductive types are in `UpperCamelCase`
- ③ Functions are named the same way as their return values

Some remarks on naming conventions

Capitalization rules:

- ① theorems, lemmas and names of proofs are in `snake_case`.
- ② structures, classes and inductive types are in `UpperCamelCase`
- ③ Functions are named the same way as their return values
- ④ All other terms of Types are in `lowerCamelCase`.

Some remarks on naming conventions

Capitalization rules:

- ① theorems, lemmas and names of proofs are in `snake_case`.
- ② structures, classes and inductive types are in `UpperCamelCase`
- ③ Functions are named the same way as their return values
- ④ All other terms of Types are in `lowerCamelCase`.
- ⑤ When something that should be in `UpperCamelCase` is part of something in `snake_case`, it is referenced in `lowerCamelCase`.

Some remarks on naming conventions

Capitalization rules:

- ① theorems, lemmas and names of proofs are in `snake_case`.
- ② structures, classes and inductive types are in `UpperCamelCase`
- ③ Functions are named the same way as their return values
- ④ All other terms of Types are in `lowerCamelCase`.
- ⑤ When something that should be in `UpperCamelCase` is part of something in `snake_case`, it is referenced in `lowerCamelCase`.
- ⑥ Acronyms (e.g. LE) are capitalized as a group

Some remarks on naming conventions

Capitalization rules:

- ① theorems, lemmas and names of proofs are in `snake_case`.
- ② structures, classes and inductive types are in `UpperCamelCase`
- ③ Functions are named the same way as their return values
- ④ All other terms of Types are in `lowerCamelCase`.
- ⑤ When something that should be in `UpperCamelCase` is part of something in `snake_case`, it is referenced in `lowerCamelCase`.
- ⑥ Acronyms (e.g. LE) are capitalized as a group
- ⑦ Fields of a structure and constructors of an inductive type are named the way the structure or inductive type would be named, respectively.

Some remarks on naming conventions

Capitalization rules:

- ① theorems, lemmas and names of proofs are in `snake_case`.
- ② structures, classes and inductive types are in `UpperCamelCase`
- ③ Functions are named the same way as their return values
- ④ All other terms of Types are in `lowerCamelCase`.
- ⑤ When something that should be in `UpperCamelCase` is part of something in `snake_case`, it is referenced in `lowerCamelCase`.
- ⑥ Acronyms (e.g. LE) are capitalized as a group
- ⑦ Fields of a structure and constructors of an inductive type are named the way the structure or inductive type would be named, respectively.

Of course, all of these rules have exceptions (see naming conventions guide for further explanations).

Examples of capitalization

```
1  -- follows rule 2
2  structure OneHom (M : Type _) (N : Type _) [One M] [One N] where
3  toFun : M → N -- follows rule 4 via rule 3 and rule 7
4  map_one' : toFun 1 = 1 -- follows rule 1 via rule 7
5
```

Some remarks on naming conventions

- „Axiomatic properties“ of an operation are put in a namespace that starts with the name of that operation:

```
1      abbrev And.elim.{u_1} {a b : Prop} {α : Sort u_1} (f : a → b → α)
2        (h : a ∧ b) : α := sorry
3
```

Some remarks on naming conventions

- „Axiomatic properties“ of an operation are put in a namespace that starts with the name of that operation:

```
1      abbrev And.elim.{u_1} {a b : Prop} {α : Sort u_1} (f : a → b → α)
2      (h : a ∧ b) : α := sorry
3
```

- This is **not** the case for axiomatic arithmetic operations:

```
1      theorem mul_comm.{u_1} {G : Type u_1} [CommMagma G] (a b : G) :
2      a * b = b * a := sorry
3
```

Some remarks on naming conventions

- „Axiomatic properties“ of an operation are put in a namespace that starts with the name of that operation:

```
1      abbrev And.elim.{u_1} {a b : Prop} {α : Sort u_1} (f : a → b → α)
2      (h : a ∧ b) : α := sorry
3
```

- This is **not** the case for axiomatic arithmetic operations:

```
1      theorem mul_comm.{u_1} {G : Type u_1} [CommMagma G] (a b : G) :
2      a * b = b * a := sorry
3
```

- Theorems are named after their conclusions

```
1      theorem Nat.succ_ne_zero (n : Nat) : n.succ ≠ 0 := sorry
2
```

Some remarks on naming conventions

- If an operation is written as an infix notation, this is reflected in the theorem name:

```
1  theorem neg_mul_neg.{u} {α : Type u} [Mul α] [HasDistribNeg α] (a b : α) :  
2    -a * -b = a * b := sorry  
3  --not mul_neg_neg  
4
```

Some remarks on naming conventions

- If an operation is written as an infix notation, this is reflected in the theorem name:

```
1 theorem neg_mul_neg.{u} {α : Type u} [Mul α] [HasDistribNeg α] (a b : α) :  
2   -a * -b = a * b := sorry  
3 --not mul_neg_neg  
4
```

- If naming the conclusion is not descriptive enough, the word „of “ is used to distinguish the conclusion from the hypothesis:

```
1 lemma lt_of_not_ge.{u_1} {α : Type u_1} [LinearOrder α] {a b : α}  
2   (h : ¬b ≤ a) : a < b := sorry  
3
```

Some remarks on naming conventions

- If an operation is written as an infix notation, this is reflected in the theorem name:

```
1 theorem neg_mul_neg.{u} {α : Type u} [Mul α] [HasDistribNeg α] (a b : α) :  
2   -a * -b = a * b := sorry  
3 --not mul_neg_neg  
4
```

- If naming the conclusion is not descriptive enough, the word „of “ is used to distinguish the conclusion from the hypothesis:

```
1 lemma lt_of_not_ge.{u_1} {α : Type u_1} [LinearOrder α] {a b : α}  
2   (h : ¬b ≤ a) : a < b := sorry  
3
```

- There is a specific dictionary for translating mathematical symbols into words, make sure to check the naming conventions guide!

Naming conventions: an example

```
1  -- this theorem does not adhere to the naming conventions, how should it be called?
2  theorem inv_is_unit_times_self_eq_1 {M : Type*} [Monoid M] {a : M} (h : IsUnit a) :
3   $\uparrow(\text{IsUnit.unit } h)^{-1} * a = 1$  := sorry
```

Naming conventions: an example

```
1  -- this theorem does not adhere to the naming conventions, how should it be called?  
2  theorem inv_is_unit_times_self_eq_1 {M : Type*} [Monoid M] {a : M} (h : IsUnit a) :  
3  ↑(IsUnit.unit h)-1 * a = 1 := sorry
```

- Write one instead of 1

Naming conventions: an example

```
1  -- this theorem does not adhere to the naming conventions, how should it be called?  
2  theorem inv_is_unit_times_self_eq_1 {M : Type*} [Monoid M] {a : M} (h : IsUnit a) :  
3  ↑(IsUnit.unit h)-1 * a = 1 := sorry
```

- Write one instead of 1
- The theorem name is already descriptive enough without the eq_one

Naming conventions: an example

```
1  -- this theorem does not adhere to the naming conventions, how should it be called?
2  theorem inv_is_unit_times_self_eq_1 {M : Type*} [Monoid M] {a : M} (h : IsUnit a) :
3  ↑(IsUnit.unit h)-1 * a = 1 := sorry
```

- Write one instead of 1
- The theorem name is already descriptive enough without the eq_one
- Write mul instead of times

Naming conventions: an example

```
1  -- this theorem does not adhere to the naming conventions, how should it be called?
2  theorem inv_is_unit_times_self_eq_1 {M : Type*} [Monoid M] {a : M} (h : IsUnit a) :
3   $\uparrow(\text{IsUnit.unit } h)^{-1} * a = 1$  := sorry
4
```

- Write one instead of 1
- The theorem name is already descriptive enough without the eq_one
- Write mul instead of times
- Reference IsUnit in UpperCamelCase and with dot-notation in the theorem name

Naming conventions: an example

```
1  -- this theorem does not adhere to the naming conventions, how should it be called?
2  theorem inv_is_unit_times_self_eq_1 {M : Type*} [Monoid M] {a : M} (h : IsUnit a) :
3  ↑(IsUnit.unit h)-1 * a = 1 := sorry
4
```

- Write one instead of 1
- The theorem name is already descriptive enough without the eq_one
- Write mul instead of times
- Reference IsUnit in UpperCamelCase and with dot-notation in the theorem name
- The coercion (namely Subtype.val) should be accounted for in the theorem name

Naming conventions: an example

```
1  -- this theorem does not adhere to the naming conventions, how should it be called?
2  theorem inv_is_unit_times_self_eq_1 {M : Type*} [Monoid M] {a : M} (h : IsUnit a) :
3  ↑(IsUnit.unit h)-1 * a = 1 := sorry
4
```

- Write one instead of 1
- The theorem name is already descriptive enough without the eq_one
- Write mul instead of times
- Reference IsUnit in UpperCamelCase and with dot-notation in the theorem name
- The coercion (namely Subtype.val) should be accounted for in the theorem name

```
1  -- Better naming:
2  theorem IsUnit.inv_val_mul {M : Type*} [Monoid M] {a : M} (h : IsUnit a) :
3  ↑(IsUnit.unit h)-1 * a = 1 := sorry
4
```

Documentation

If you are adding files to mathlib, make sure they start with

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header
 - ▶ A summary of the contents

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header
 - ▶ A summary of the contents
 - ▶ A section titled *Main definitions* (optional)

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header
 - ▶ A summary of the contents
 - ▶ A section titled *Main definitions* (optional)
 - ▶ A section titled *Main statements* (optional)

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header
 - ▶ A summary of the contents
 - ▶ A section titled *Main definitions* (optional)
 - ▶ A section titled *Main statements* (optional)
 - ▶ A section titled *Notation* (if new notation is introduced)

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header
 - ▶ A summary of the contents
 - ▶ A section titled *Main definitions* (optional)
 - ▶ A section titled *Main statements* (optional)
 - ▶ A section titled *Notation* (if new notation is introduced)
 - ▶ A section titled *Implementation notes*

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header
 - ▶ A summary of the contents
 - ▶ A section titled *Main definitions* (optional)
 - ▶ A section titled *Main statements* (optional)
 - ▶ A section titled *Notation* (if new notation is introduced)
 - ▶ A section titled *Implementation notes*
 - ▶ A section titled *References*

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header
 - ▶ A summary of the contents
 - ▶ A section titled *Main definitions* (optional)
 - ▶ A section titled *Main statements* (optional)
 - ▶ A section titled *Notation* (if new notation is introduced)
 - ▶ A section titled *Implementation notes*
 - ▶ A section titled *References*
 - ▶ A list of *Tags*

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header
 - ▶ A summary of the contents
 - ▶ A section titled *Main definitions* (optional)
 - ▶ A section titled *Main statements* (optional)
 - ▶ A section titled *Notation* (if new notation is introduced)
 - ▶ A section titled *Implementation notes*
 - ▶ A section titled *References*
 - ▶ A list of *Tags*

Moreover, make sure every **definition** and every **main theorem** has a *descriptive doc string*;

Documentation

If you are adding files to mathlib, make sure they start with

- A **header** with copyright information
- the list of **imports**
- A **module docstring** containing:
 - ▶ A first-level header
 - ▶ A summary of the contents
 - ▶ A section titled *Main definitions* (optional)
 - ▶ A section titled *Main statements* (optional)
 - ▶ A section titled *Notation* (if new notation is introduced)
 - ▶ A section titled *Implementation notes*
 - ▶ A section titled *References*
 - ▶ A list of *Tags*

Moreover, make sure every **definition** and every **main theorem** has a *descriptive doc string*; **long proofs** should contain *comments* describing the underlying thought process.

What does a proper documentation look like?

```
1  /-
2  Copyright (c) 2018 Robert Y. Lewis. All rights reserved.
3  Released under Apache 2.0 license as described in the file LICENSE.
4  Authors: Robert Y. Lewis
5  -/
6  module
7
8  public import Mathlib.Algebra.Order.AbsoluteValue.Basic
9  public import Mathlib.NumberTheory.Padics.PadicVal.Basic
10
11  /-!
12  # p-adic norm
13
14  This file defines the 'p'-adic norm on ' $\mathbb{Q}$ '.
15  -/
16
```

What does a proper documentation look like?

```
1  /-!  
2  The 'p'-adic valuation on ' $\mathbb{Q}$ ' is the difference of the multiplicities of 'p' in  
3  the numerator and  
4  denominator of 'q'. This function obeys the standard properties of a valuation,  
5  with the appropriate  
6  assumptions on 'p'.  
7  
8  The valuation induces a norm on ' $\mathbb{Q}$ '. This norm is a nonarchimedean absolute value.  
9  It takes values in  $\{0\} \cup \{1/p^k \mid k \in \mathbb{Z}\}$ .  
10  
11 ## Implementation notes  
12  
13 Much, but not all, of this file assumes that 'p' is prime. This assumption is  
14 inferred automatically  
by taking '[Fact p.Prime]' as a type class argument.  
-/
```

What does a proper documentation look like?

```
1  /-!  
2  ## References  
3  
4  * [F. Q. Gouvêa, *p-adic numbers*][gouvea1997]  
5  * [R. Y. Lewis, *A formal proof of Hensel's lemma over the p-adic  
6    integers*][lewis2019]  
7  * <https://en.wikipedia.org/wiki/P-adic\_number>  
8  
9  ## Tags  
10  
11  p-adic, p adic, padic, norm, valuation  
12  -/
```


Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Mathlib's current policy on AI usage

Mathlib's current policy on AI usage

- Usage of AI for helping with code generation is not forbidden, but **must be clearly indicated**

Mathlib's current policy on AI usage

- Usage of AI for helping with code generation is not forbidden, but **must be clearly indicated**
 - ▶ Mention your AI-usage in the description of the PR. In particular, mention which AI you used & how you used it.

Mathlib's current policy on AI usage

- Usage of AI for helping with code generation is not forbidden, but **must be clearly indicated**
 - ▶ Mention your AI-usage in the description of the PR. In particular, mention which AI you used & how you used it.
 - ▶ The label LLM-generated must be added to any PR with a significant amount of AI-generated code

Mathlib's current policy on AI usage

- Usage of AI for helping with code generation is not forbidden, but **must be clearly indicated**
 - ▶ Mention your AI-usage in the description of the PR. In particular, mention which AI you used & how you used it.
 - ▶ The label LLM-generated must be added to any PR with a significant amount of AI-generated code
 - ▶ It is crucial that you fully understand your own pull request!

Mathlib's current policy on AI usage

- Usage of AI for helping with code generation is not forbidden, but **must be clearly indicated**
 - ▶ Mention your AI-usage in the description of the PR. In particular, mention which AI you used & how you used it.
 - ▶ The label LLM-generated must be added to any PR with a significant amount of AI-generated code
 - ▶ It is crucial that you fully understand your own pull request!
- Low-quality LLM-generated pull requests will be closed without comment.

Mathlib's current policy on AI usage

- Usage of AI for helping with code generation is not forbidden, but **must be clearly indicated**
 - ▶ Mention your AI-usage in the description of the PR. In particular, mention which AI you used & how you used it.
 - ▶ The label LLM-generated must be added to any PR with a significant amount of AI-generated code
 - ▶ It is crucial that you fully understand your own pull request!
- Low-quality LLM-generated pull requests will be closed without comment.
- **Do not** use generative AI for writing comments on Zulip or GitHub!

Mathlib's AI policy: Challenges & possible future changes

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas
 - ★ **Level 2** : Human coded with minor assists

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas
 - ★ **Level 2** : Human coded with minor assists
 - ★ **Level 3** : Human coded, AI assisted nontrivially

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas
 - ★ **Level 2** : Human coded with minor assists
 - ★ **Level 3** : Human coded, AI assisted nontrivially
 - ★ **Level 4** : Human coded, AI helped significantly

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas
 - ★ **Level 2** : Human coded with minor assists
 - ★ **Level 3** : Human coded, AI assisted nontrivially
 - ★ **Level 4** : Human coded, AI helped significantly
 - ★ **Level 5** : AI coded, human checked & understood everything

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas
 - ★ **Level 2** : Human coded with minor assists
 - ★ **Level 3** : Human coded, AI assisted nontrivially
 - ★ **Level 4** : Human coded, AI helped significantly
 - ★ **Level 5** : AI coded, human checked & understood everything
- What does *fully understanding your code* mean & how to check it?

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas
 - ★ **Level 2** : Human coded with minor assists
 - ★ **Level 3** : Human coded, AI assisted nontrivially
 - ★ **Level 4** : Human coded, AI helped significantly
 - ★ **Level 5** : AI coded, human checked & understood everything
- What does *fully understanding your code* mean & how to check it?
 - ▶ Problem: often insufficient disclosure of AI use & the statement „I have read the code and can vouch for every line“ is added without real understanding

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas
 - ★ **Level 2** : Human coded with minor assists
 - ★ **Level 3** : Human coded, AI assisted nontrivially
 - ★ **Level 4** : Human coded, AI helped significantly
 - ★ **Level 5** : AI coded, human checked & understood everything
- What does *fully understanding your code* mean & how to check it?
 - ▶ Problem: often insufficient disclosure of AI use & the statement „I have read the code and can vouch for every line“ is added without real understanding
 - ▶ *Full understanding* might have very different interpretations

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas
 - ★ **Level 2** : Human coded with minor assists
 - ★ **Level 3** : Human coded, AI assisted nontrivially
 - ★ **Level 4** : Human coded, AI helped significantly
 - ★ **Level 5** : AI coded, human checked & understood everything
- What does *fully understanding your code* mean & how to check it?
 - ▶ Problem: often insufficient disclosure of AI use & the statement „I have read the code and can vouch for every line“ is added without real understanding
 - ▶ *Full understanding* might have very different interpretations
 - ▶ ⇒ As of now: no satisfying solution

Mathlib's AI policy: Challenges & possible future changes

- What does *describe how you used it* mean?
 - ▶ As of now: no clear definition
 - ▶ Possible implementation in the future: Self-assessed AI level for contributions
 - ★ **Level 0** : No AI input
 - ★ **Level 1** : Human asked Chatbot for ideas
 - ★ **Level 2** : Human coded with minor assists
 - ★ **Level 3** : Human coded, AI assisted nontrivially
 - ★ **Level 4** : Human coded, AI helped significantly
 - ★ **Level 5** : AI coded, human checked & understood everything
- What does *fully understanding your code* mean & how to check it?
 - ▶ Problem: often insufficient disclosure of AI use & the statement „I have read the code and can vouch for every line“ is added without real understanding
 - ▶ *Full understanding* might have very different interpretations
 - ▶ ⇒ As of now: no satisfying solution
- Check PSA: `mathlib` AI policy on Zulip for current state of the debate

Mathlib's AI policy: Reasons

Mathlib's AI policy: Reasons

AI makes different mistakes than humans

Mathlib's AI policy: Reasons

AI makes different mistakes than humans

Reviewing pull requests takes time & effort

Mathlib's AI policy: Reasons

AI makes different mistakes than humans

Reviewing pull requests takes time & effort → Review process is supposed to have an educating effect

Mathlib's AI policy: Reasons

AI makes different mistakes than humans

Reviewing pull requests takes time & effort → Review process is supposed to have an educating effect

Writing comments yourself is a matter of respect

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Contribution Process

Maintaining and version control of Mathlib are done using `git` on the Platform GitHub.

Contribution Process

Maintaining and version control of Mathlib are done using `git` on the Platform GitHub.

Working with `git` for the first time can seem scary - but don't worry, there's a lot of helpful resources!

Contribution Process

Maintaining and version control of Mathlib are done using `git` on the Platform GitHub.

Working with `git` for the first time can seem scary - but don't worry, there's a lot of helpful resources!

Check *git guide for Mathlib4 contributors*

Contribution Process

Maintaining and version control of Mathlib are done using `git` on the Platform GitHub.

Working with `git` for the first time can seem scary - but don't worry, there's a lot of helpful resources!

Check *git guide for Mathlib4 contributors* → really helpful if you're inexperienced with `git` (trust me!)

Working on Mathlib

One-time setup:

Working on Mathlib

One-time setup:

- 1 Fork the Mathlib repository.

Working on Mathlib

One-time setup:

- 1 Fork the Mathlib repository.
- 2 Make a local clone of your fork and configure it properly (*check the instructions!*)

Working on Mathlib

One-time setup:

- 1 Fork the Mathlib repository.
- 2 Make a local clone of your fork and configure it properly (*check the instructions!*)

Daily workflow:

Working on Mathlib

One-time setup:

- 1 Fork the Mathlib repository.
- 2 Make a local clone of your fork and configure it properly (*check the instructions!*)

Daily workflow:

- 1 Create a new branch by using the command `git switch -c`

Working on Mathlib

One-time setup:

- 1 Fork the Mathlib repository.
- 2 Make a local clone of your fork and configure it properly (*check the instructions!*)

Daily workflow:

- 1 Create a new branch by using the command `git switch -c`
- 2 Make local changes (e.g. in VSCode)

Working on Mathlib

One-time setup:

- 1 Fork the Mathlib repository.
- 2 Make a local clone of your fork and configure it properly (*check the instructions!*)

Daily workflow:

- 1 Create a new branch by using the command `git switch -c`
- 2 Make local changes (e.g. in VSCode)
- 3 Commit your changes

Working on Mathlib

One-time setup:

- 1 Fork the Mathlib repository.
- 2 Make a local clone of your fork and configure it properly (*check the instructions!*)

Daily workflow:

- 1 Create a new branch by using the command `git switch -c`
- 2 Make local changes (e.g. in VSCode)
- 3 Commit your changes
- 4 Make sure everything works by running `lake build` (and `lake exe mk_all` if you created new files) [optional, but encouraged]

Working on Mathlib

One-time setup:

- 1 Fork the Mathlib repository.
- 2 Make a local clone of your fork and configure it properly (*check the instructions!*)

Daily workflow:

- 1 Create a new branch by using the command `git switch -c`
- 2 Make local changes (e.g. in VSCode)
- 3 Commit your changes
- 4 Make sure everything works by running `lake build` (and `lake exe mk_all` if you created new files) [optional, but encouraged]
- 5 push your changes back on GitHub

Opening a Pull request

Opening a Pull request

- Go to your fork on GitHub

Opening a Pull request

- Go to your fork on GitHub
- Click „Compare & pull request“

Opening a Pull request

- Go to your fork on GitHub
- Click „Compare & pull request“
- Fill in pull request title & description (*Make sure to adhere to the guidelines!*)

Opening a Pull request

- Go to your fork on GitHub
- Click „Compare & pull request“
- Fill in pull request title & description (*Make sure to adhere to the guidelines!*)
- Click „Create pull request“

Opening a Pull request

- Go to your fork on GitHub
- Click „Compare & pull request“
- Fill in pull request title & description (*Make sure to adhere to the guidelines!*)
- Click „Create pull request“
- Wait for CI-tests to finish & address potential issues

Opening a Pull request

- Go to your fork on GitHub
- Click „Compare & pull request“
- Fill in pull request title & description (*Make sure to adhere to the guidelines!*)
- Click „Create pull request“
- Wait for CI-tests to finish & address potential issues

General guidelines:

Opening a Pull request

- Go to your fork on GitHub
- Click „Compare & pull request“
- Fill in pull request title & description (*Make sure to adhere to the guidelines!*)
- Click „Create pull request“
- Wait for CI-tests to finish & address potential issues

General guidelines:

- The smaller, the better

Opening a Pull request

- Go to your fork on GitHub
- Click „Compare & pull request“
- Fill in pull request title & description (*Make sure to adhere to the guidelines!*)
- Click „Create pull request“
- Wait for CI-tests to finish & address potential issues

General guidelines:

- The smaller, the better
- Introduce yourself on Zulip & link your pull request!

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**
- **fix**

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**
- **fix**
- **doc**

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**
- **fix**
- **doc**
- **style**

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**
- **fix**
- **doc**
- **style**
- **refactor**

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**
- **fix**
- **doc**
- **style**
- **refactor**
- **test**

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**
- **fix**
- **doc**
- **style**
- **refactor**
- **test**
- **chore**

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**
- **fix**
- **doc**
- **style**
- **refactor**
- **test**
- **chore**
- **perf**

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**
- **fix**
- **doc**
- **style**
- **refactor**
- **test**
- **chore**
- **perf**
- **ci**

Pull request title

The title should be of the form:

`<type>(<optional-scope>): <subject>`

`<type>` should be one of the following:

- **feat**
- **fix**
- **doc**
- **style**
- **refactor**
- **test**
- **chore**
- **perf**
- **ci**

`<optional-scope>` should specify the modified modules (if not immediately clear from `<subject>`)

Commit conventions

Conventions for <subject>

Commit conventions

Conventions for <subject>

- use imperative, present tense („change“ insted of „changes“ or „changed“)

Commit conventions

Conventions for <subject>

- use imperative, present tense („change“ insted of „changes“ or „changed“)
- Write everything in lower case

Commit conventions

Conventions for <subject>

- use imperative, present tense („change“ insted of „changes“ or „changed“)
- Write everything in lower case
- No dot at the end

Commit conventions

Conventions for <subject>

- use imperative, present tense („change“ insted of „changes“ or „changed“)
- Write everything in lower case
- No dot at the end

Structure of the description:

Commit conventions

Conventions for <subject>

- use imperative, present tense („change“ insted of „changes“ or „changed“)
- Write everything in lower case
- No dot at the end

Structure of the description:

```
<body>  
<NEWLINE>  
<footer>  
<NEWLINE>  
<dependencies>
```

Commit conventions

Conventions for <body>:

Commit conventions

Conventions for <body>:

- Grammar as in <subject>

Commit conventions

Conventions for <body>:

- Grammar as in <subject>
- Include motivation & explain what behaviours have changed!

Commit conventions

Conventions for <body>:

- Grammar as in <subject>
- Include motivation & explain what behaviours have changed!

Conventions for <footer>:

Commit conventions

Conventions for <body>:

- Grammar as in <subject>
- Include motivation & explain what behaviours have changed!

Conventions for <footer>:

- Optional

Commit conventions

Conventions for <body>:

- Grammar as in <subject>
- Include motivation & explain what behaviours have changed!

Conventions for <footer>:

- Optional
- Address potential side effects (if any) & justify!

Commit conventions

Conventions for <body>:

- Grammar as in <subject>
- Include motivation & explain what behaviours have changed!

Conventions for <footer>:

- Optional
- Address potential side effects (if any) & justify!
- Reference bugs closed by your PR (each on separate line, prefixed by „Closes“)

Commit conventions

Conventions for <body>:

- Grammar as in <subject>
- Include motivation & explain what behaviours have changed!

Conventions for <footer>:

- Optional
- Address potential side effects (if any) & justify!
- Reference bugs closed by your PR (each on separate line, prefixed by „Closes“)

Conventions for <dependencies>

Commit conventions

Conventions for <body>:

- Grammar as in <subject>
- Include motivation & explain what behaviours have changed!

Conventions for <footer>:

- Optional
- Address potential side effects (if any) & justify!
- Reference bugs closed by your PR (each on separate line, prefixed by „Closes“)

Conventions for <dependencies>

- List as follows: – [] depends on: #XXXX

Commit conventions

Conventions for <body>:

- Grammar as in <subject>
- Include motivation & explain what behaviours have changed!

Conventions for <footer>:

- Optional
- Address potential side effects (if any) & justify!
- Reference bugs closed by your PR (each on separate line, prefixed by „Closes“)

Conventions for <dependencies>

- List as follows: – [] depends on: #XXXX

If you have moved or deleted any declarations, add --- at the end of your commit message and list „Moves“ and „Deletions“ as bullet point lists afterwards.

Examples of good pull request descriptions:

```
doc(CategoryTheory/EssentialImage): typo and punctuation
```

```
Fix a typo, add two periods.
```

Examples of good pull request descriptions

feat: The norm on 'Unitization' is a C*-norm

This shows that C*-algebras are always 'RegularNormedAlgebra's, so that their 'Unitization' is equipped with a norm. Moreover, we show this norm is a C*-norm.

- [] depends on: #5330
- [] depends on: #5741
- [] depends on: #5742
- [] depends on: #5743

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Lifecycle of a PR

Lifecycle of a PR

- 1 CI (continuous integration) runs test on your code

Lifecycle of a PR

- ④ CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style

Lifecycle of a PR

- ④ CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and `sorrrys`

Lifecycle of a PR

- ④ CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and `sorrrys`
 - ▶ etc

Lifecycle of a PR

- 1 CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrrys
 - ▶ etc
- 2 If your PR passes CI, anybody can review your code

Lifecycle of a PR

- ① CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrrys
 - ▶ etc
- ② If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes

Lifecycle of a PR

- ① CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrrys
 - ▶ etc
- ② If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add awaiting-author label

Lifecycle of a PR

- ① CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrrys
 - ▶ etc
- ② If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add awaiting-author label
- ③ Your turn:

Lifecycle of a PR

- ① CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and `sorrrys`
 - ▶ etc
- ② If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add `awaiting-author` label
- ③ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)

Lifecycle of a PR

- ① CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrys
 - ▶ etc
- ② If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add awaiting-author label
- ③ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)
 - ▶ delete awaiting-author label

Lifecycle of a PR

- ① CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrys
 - ▶ etc
- ② If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add awaiting-author label
- ③ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)
 - ▶ delete awaiting-author label
- ④ Next review round:

Lifecycle of a PR

- ❶ CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and `sorrrys`
 - ▶ etc
- ❷ If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add `awaiting-author` label
- ❸ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)
 - ▶ delete `awaiting-author` label
- ❹ Next review round:
 - ▶ Possibly more suggested changes

Lifecycle of a PR

- ❶ CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrrys
 - ▶ etc
- ❷ If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add `awaiting-author` label
- ❸ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)
 - ▶ delete `awaiting-author` label
- ❹ Next review round:
 - ▶ Possibly more suggested changes
 - ▶ If people think your PR is ready to merge, they can add an „approving review“ (or a `maintainer-merge-label`)

Lifecycle of a PR

- ① CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrrys
 - ▶ etc
- ② If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add awaiting-author label
- ③ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)
 - ▶ delete awaiting-author label
- ④ Next review round:
 - ▶ Possibly more suggested changes
 - ▶ If people think your PR is ready to merge, they can add an „approving review“ (or a maintainer-merge-label)
- ⑤ Only *maintainers* can merge your PR

Lifecycle of a PR

- ❶ CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrrys
 - ▶ etc
- ❷ If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add awaiting-author label
- ❸ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)
 - ▶ delete awaiting-author label
- ❹ Next review round:
 - ▶ Possibly more suggested changes
 - ▶ If people think your PR is ready to merge, they can add an „approving review“ (or a maintainer-merge-label)
- ❺ Only *maintainers* can merge your PR
 - ▶ They add ready-to-merge-label

Lifecycle of a PR

- ① CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and sorrys
 - ▶ etc
- ② If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add `awaiting-author` label
- ③ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)
 - ▶ delete `awaiting-author` label
- ④ Next review round:
 - ▶ Possibly more suggested changes
 - ▶ If people think your PR is ready to merge, they can add an „approving review“ (or a `maintainer-merge-label`)
- ⑤ Only *maintainers* can merge your PR
 - ▶ They add `ready-to-merge-label`
 - ▶ Bot **bors** merges PR automatically

Lifecycle of a PR

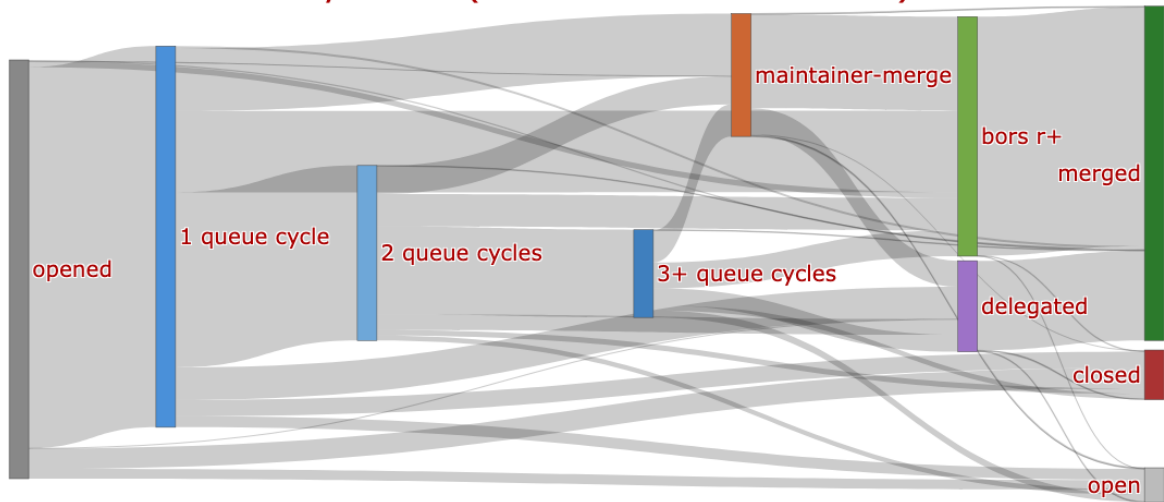
- ❶ CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and `sorrys`
 - ▶ etc
- ❷ If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add `awaiting-author` label
- ❸ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)
 - ▶ delete `awaiting-author` label
- ❹ Next review round:
 - ▶ Possibly more suggested changes
 - ▶ If people think your PR is ready to merge, they can add an „approving review“ (or a `maintainer-merge-label`)
- ❺ Only *maintainers* can merge your PR
 - ▶ They add `ready-to-merge-label`
 - ▶ Bot **bors** merges PR automatically
 - ▶ Alternatively, maintainers *delegate* the merging to you (type **bors merge** yourself if ready)

Lifecycle of a PR

- ❶ CI (continuous integration) runs test on your code
 - ▶ tests correct title formatting & style
 - ▶ checks if everything compiles without errors and `sorrys`
 - ▶ etc
- ❷ If your PR passes CI, anybody can review your code
 - ▶ they will suggest changes
 - ▶ they will add `awaiting-author` label
- ❸ Your turn:
 - ▶ address their comments (by adopting their commits or justifying)
 - ▶ delete `awaiting-author` label
- ❹ Next review round:
 - ▶ Possibly more suggested changes
 - ▶ If people think your PR is ready to merge, they can add an „approving review“ (or a `maintainer-merge-label`)
- ❺ Only *maintainers* can merge your PR
 - ▶ They add `ready-to-merge-label`
 - ▶ Bot **bors** merges PR automatically
 - ▶ Alternatively, maintainers *delegate* the merging to you (type **bors merge** yourself if ready)
- ❻ Congratulations, your suggested change was adopted in Mathlib!

The lifecycle of a PR

mathlib4 PR lifecycle flow (2024-02-15 to 2026-05-21)



Credits: Bryan Chen

Oh no, a merge conflict!

Merge conflicts arise when your PR conflicts with a recent change in Mathlib (i.e. another PR that modified the same definitions / theorems as yours got adopted first)

Oh no, a merge conflict!

Merge conflicts arise when your PR conflicts with a recent change in Mathlib (i.e. another PR that modified the same definitions / theorems as yours got adopted first)

If a merge conflict arises, a **merge-conflict** label is automatically applied to your PR.

Oh no, a merge conflict!

Merge conflicts arise when your PR conflicts with a recent change in Mathlib (i.e. another PR that modified the same definitions / theorems as yours got adopted first)

If a merge conflict arises, a **merge-conflict** label is automatically applied to your PR.

This temporarily hides your PR to reviewers, you should resolve the conflict yourself.

Oh no, a merge conflict!

Merge conflicts arise when your PR conflicts with a recent change in Mathlib (i.e. another PR that modified the same definitions / theorems as yours got adopted first)

If a merge conflict arises, a **merge-conflict** label is automatically applied to your PR.

This temporarily hides your PR to reviewers, you should resolve the conflict yourself.

Check the *GitHub tutorial on merge conflicts* for advice on how to do this.

Labels

So-called „labels“ (e.g. `awaiting-author`, `maintainer-merge`, `ready-to-merge`) indicate the status of the review process.

Labels

So-called „labels“ (e.g. `awaiting-author`, `maintainer-merge`, `ready-to-merge`) indicate the status of the review process.

Purpose of labels: convey information about your PR's content, indicate the progress & guide the review process.

Labels

So-called „labels“ (e.g. `awaiting-author`, `maintainer-merge`, `ready-to-merge`) indicate the status of the review process.

Purpose of labels: convey information about your PR's content, indicate the progress & guide the review process.

You can add labels by posting a comment with the name of the label on your own PR (e.g. „LLM-generated“). You can remove the labels by posting a comment with a „-“ before the name of the label (e.g. „-awaiting-author“).

Labels

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy
- help-wanted, please-adopt

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy
- help-wanted, please-adopt
- RFC: Stands for „Request for Comment“

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy
- help-wanted, please-adopt
- RFC: Stands for „Request for Comment“
- LLM-generated

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy
- help-wanted, please-adopt
- RFC: Stands for „Request for Comment“
- LLM-generated
- brownian, carleson, CFT, FLT, infinity-cosmos, sphere-packing, toric

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy
- help-wanted, please-adopt
- RFC: Stands for „Request for Comment“
- LLM-generated
- brownian, carleson, CFT, FLT, infinity-cosmos, sphere-packing, toric
- maintainer-merge

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy
- help-wanted, please-adopt
- RFC: Stands for „Request for Comment“
- LLM-generated
- brownian, carleson, CFT, FLT, infinity-cosmos, sphere-packing, toric
- maintainer-merge
- ready-to-merge

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy
- help-wanted, please-adopt
- RFC: Stands for „Request for Comment“
- LLM-generated
- brownian, carleson, CFT, FLT, infinity-cosmos, sphere-packing, toric
- maintainer-merge
- ready-to-merge
- delegated

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy
- help-wanted, please-adopt
- RFC: Stands for „Request for Comment“
- LLM-generated
- brownian, carleson, CFT, FLT, infinity-cosmos, sphere-packing, toric
- maintainer-merge
- ready-to-merge
- delegated
- merge-conflict

Labels

- *topic labels* (e.g. t-topology, t-category-theory, t-analysis)
- awaiting-author
- awaiting-zulip
- WIP: Stands for „Work in Progress“
- easy
- help-wanted, please-adopt
- RFC: Stands for „Request for Comment“
- LLM-generated
- brownian, carleson, CFT, FLT, infinity-cosmos, sphere-packing, toric
- maintainer-merge
- ready-to-merge
- delegated
- merge-conflict
- blocked-by-other-PR

Some examples

- <https://github.com/leanprover-community/mathlib4/pull/39996>
- <https://github.com/leanprover-community/mathlib4/pull/5580>

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

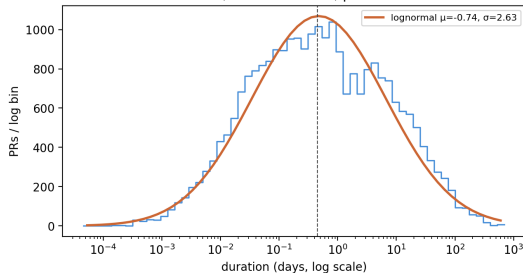
4.2 Review guidelines

4.3 Queueboard

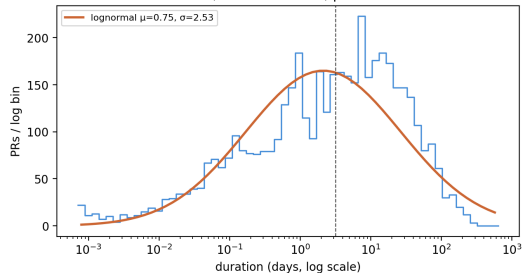
How long does each stage take?

Per-stage duration distributions (log bins + lognormal fit)

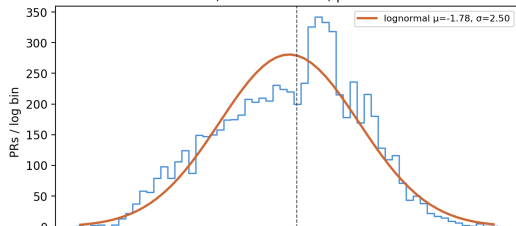
open → first review
n=25648, median=0.45d, p90=16.62d



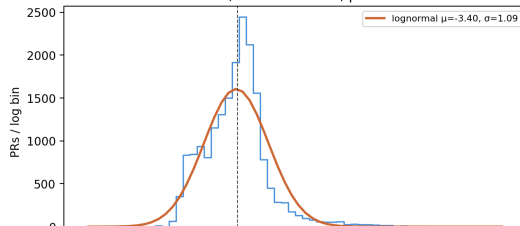
first review → maintainer-merge (iterated, ≥ 60 s)
n=4571, median=3.18d, p90=39.67d



MM → ready-to-merge
n=6985, median=0.22d, p90=3.80d

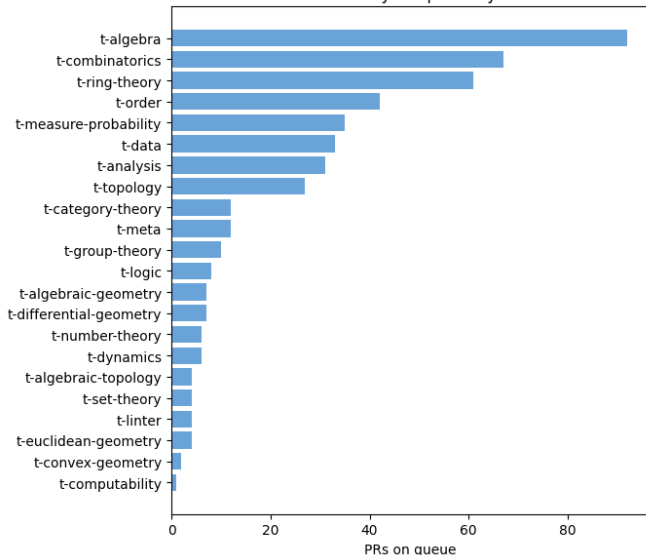


RTM → merged
n=18542, median=0.03d, p90=0.10d



Variation between areas

PRs currently on queue by area



Median total queue time by area

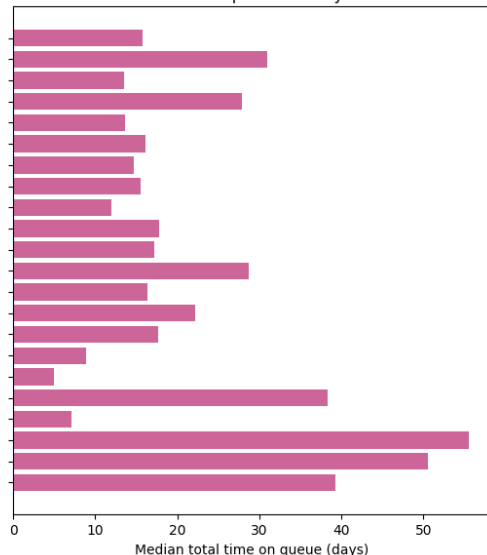


Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Different roles

Maintainers:

Different roles

Maintainers:

- Currently 28 people

Different roles

Maintainers:

- Currently 28 people
- Only role with the right to merge PRs to mathlib

Different roles

Maintainers:

- Currently 28 people
- Only role with the right to merge PRs to mathlib
- Do not have the time to review every PR manually

Different roles

Maintainers:

- Currently 28 people
- Only role with the right to merge PRs to mathlib
- Do not have the time to review every PR manually

Anyone...

Different roles

Maintainers:

- Currently 28 people
- Only role with the right to merge PRs to mathlib
- Do not have the time to review every PR manually

Anyone...

- ...can review pull requests, write comments & suggest changes

Different roles

Maintainers:

- Currently 28 people
- Only role with the right to merge PRs to mathlib
- Do not have the time to review every PR manually

Anyone...

- ...can review pull requests, write comments & suggest changes
- ...can leave an approving review to let the maintainers know you think the PR is ready to merge

Different roles

Maintainers:

- Currently 28 people
- Only role with the right to merge PRs to mathlib
- Do not have the time to review every PR manually

Anyone...

- ...can review pull requests, write comments & suggest changes
- ...can leave an approving review to let the maintainers know you think the PR is ready to merge
- Please read the *review guidelines* before commenting!

Different roles

Reviewers...

Different roles

Reviewers...

- ...are experienced community members who have issued many helpful reviews in the past

Different roles

Reviewers...

- ...are experienced community members who have issued many helpful reviews in the past
- ...exist in order to unburden the maintainers

Different roles

Reviewers...

- ...are experienced community members who have issued many helpful reviews in the past
- ...exist in order to unburden the maintainers
- ...can add the `maintainer-merge` label if they think a PR is ready to merge

Different roles

Reviewers...

- ...are experienced community members who have issued many helpful reviews in the past
- ...exist in order to unburden the maintainers
- ...can add the `maintainer-merge` label if they think a PR is ready to merge → higher priority for maintainers

Different roles

Reviewers...

- ...are experienced community members who have issued many helpful reviews in the past
- ...exist in order to unburden the maintainers
- ...can add the `maintainer-merge` label if they think a PR is ready to merge → higher priority for maintainers
- As of now: around 30 additional reviewers (every maintainer is also a reviewer)

The maintainers need you!

The maintainers need you!

- There are 2587 open PRs as of the making of these slides, 448 of which are awaiting review

The maintainers need you!

- There are 2587 open PRs as of the making of these slides, 448 of which are awaiting review
- Most reviewers / maintainers are working on Mathlib as a side project

The maintainers need you!

- There are 2587 open PRs as of the making of these slides, 448 of which are awaiting review
- Most reviewers / maintainers are working on Mathlib as a side project
- Some areas are already at maximum capacity

The maintainers need you!

- There are 2587 open PRs as of the making of these slides, 448 of which are awaiting review
- Most reviewers / maintainers are working on Mathlib as a side project
- Some areas are already at maximum capacity
- Aim: „leave no PR behind“

The maintainers need you!

- There are 2587 open PRs as of the making of these slides, 448 of which are awaiting review
- Most reviewers / maintainers are working on Mathlib as a side project
- Some areas are already at maximum capacity
- Aim: „leave no PR behind“ → cannot always be ensured

The maintainers need you!

- There are 2587 open PRs as of the making of these slides, 448 of which are awaiting review
- Most reviewers / maintainers are working on Mathlib as a side project
- Some areas are already at maximum capacity
- Aim: „leave no PR behind“ → cannot always be ensured

⇒ We need you! Anybody can help reviewing!

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

Review guidelines

Review guidelines

- Be nice, be humble, be respectful.

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?
- Is the author a human?

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?
- Is the author a human?
- Are the definitions „correct“ and do they come **with API**?

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?
- Is the author a human?
- Are the definitions „correct“ and do they come **with API**?
- Are the theorems appropriate?

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?
- Is the author a human?
- Are the definitions „correct“ and do they come **with API**?
- Are the theorems appropriate?
- Is the naming correct? Is the code style good?

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?
- Is the author a human?
- Are the definitions „correct“ and do they come **with API**?
- Are the theorems appropriate?
- Is the naming correct? Is the code style good?
- Is the code maintainable? Are the proofs readable? Are things in the right files?

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?
- Is the author a human?
- Are the definitions „correct“ and do they come **with API**?
- Are the theorems appropriate?
- Is the naming correct? Is the code style good?
- Is the code maintainable? Are the proofs readable? Are things in the right files?
- Is the code performant?

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?
- Is the author a human?
- Are the definitions „correct“ and do they come **with API**?
- Are the theorems appropriate?
- Is the naming correct? Is the code style good?
- Is the code maintainable? Are the proofs readable? Are things in the right files?
- Is the code performant?
 - ▶ `!bench`

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?
- Is the author a human?
- Are the definitions „correct“ and do they come **with API**?
- Are the theorems appropriate?
- Is the naming correct? Is the code style good?
- Is the code maintainable? Are the proofs readable? Are things in the right files?
- Is the code performant?
 - ▶ `!bench`
- Are automation annotations in place?

Review guidelines

- Be nice, be humble, be respectful.
- Does it belong into Mathlib?
- Is the author a human?
- Are the definitions „correct“ and do they come **with API**?
- Are the theorems appropriate?
- Is the naming correct? Is the code style good?
- Is the code maintainable? Are the proofs readable? Are things in the right files?
- Is the code performant?
 - ▶ `!bench`
- Are automation annotations in place?
- Does the code generate diamonds?

Table of Contents

1. The Review Process

2. What to contribute?

2.1 The scope of Mathlib

2.2 Mathlib's values

2.3 Style

2.4 AI policy

3. How to contribute?

3.1 Contribution Process

3.2 Lifecycle of a PR

3.3 Statistics

4. Behind the scenes

4.1 Different roles

4.2 Review guidelines

4.3 Queueboard

A useful tool: the queueboard

- <https://leanprover-community.github.io/queueboard/>

A useful tool: the queueboard

- <https://leanprover-community.github.io/queueboard/>
- Tool to keep track of the status of open pull requests

A useful tool: the queueboard

- <https://leanprover-community.github.io/queueboard/>
- Tool to keep track of the status of open pull requests
- *Review queue* with all pull requests that are ready to review

A useful tool: the queueboard

- <https://leanprover-community.github.io/queueboard/>
- Tool to keep track of the status of open pull requests
- *Review queue* with all pull requests that are ready to review
- Algorithm assigns reviewers to new PRs, respecting current workload and area preferences (opt-out possible)

Some neat features of the queueboard

Some neat features of the queueboard

- Check if your PR is in the review queue (and if not, why)

Some neat features of the queueboard

- Check if your PR is in the review queue (and if not, why)
- Search for the PRs of a specific author

Some neat features of the queueboard

- Check if your PR is in the review queue (and if not, why)
- Search for the PRs of a specific author
- Search for the PRs modifying a specific file

Some neat features of the queueboard

- Check if your PR is in the review queue (and if not, why)
- Search for the PRs of a specific author
- Search for the PRs modifying a specific file
- Search for the PRs with a specific label

Some neat features of the queueboard

- Check if your PR is in the review queue (and if not, why)
- Search for the PRs of a specific author
- Search for the PRs modifying a specific file
- Search for the PRs with a specific label
- Find out which areas are missing reviewers

Some neat features of the queueboard

- Check if your PR is in the review queue (and if not, why)
- Search for the PRs of a specific author
- Search for the PRs modifying a specific file
- Search for the PRs with a specific label
- Find out which areas are missing reviewers
- etc

Useful resources

- **Mathematical interests of the Mathlib maintainers:** <https://github.com/leanprover-community/mathlib4?tab=readme-ov-file#maintainers>
- **CSLib, Lean's computer science library:** <https://github.com/leanprover/cslib>
- **Physlib, Lean's physics library:**
<https://github.com/leanprover-community/physlib/tree/master/Physlib>
- **List of standalone projects depending on Mathlib:**
<https://reservoir.lean-lang.org/@leanprover-community/mathlib/dependents>
- **Combinatorial game theory repository:** <https://github.com/vihdzp/combinatorial-games>
- **List of Mathlib issues:** <https://github.com/leanprover-community/mathlib4/issues>
- **List of undergraduate topics missing in Mathlib:**
https://leanprover-community.github.io/undergrad_todo.html
- **Mathlib's Code of Conduct:** https://github.com/leanprover-community/mathlib4/blob/master/CODE_OF_CONDUCT.md
- **Discussion on the Mathlib AI policy:** <https://leanprover.zulipchat.com/#narrow/channel/113488-general/topic/PSA.3A.20mathlib.20AI.20policy/>

Useful resources

- **Mathlib style guide:** <https://leanprover-community.github.io/contribute/style.html>
- **Mathlib naming conventions:**
<https://leanprover-community.github.io/contribute/naming.html>
- **Documentation style guide for Mathlib:**
<https://leanprover-community.github.io/contribute/doc.html>
- **Git Guide for Mathlib4 Contributors:**
<https://leanprover-community.github.io/contribute/git.html>
- **Commit conventions:**
<https://leanprover-community.github.io/contribute/commit.html>
- **Dealing with merge conflicts:**
<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github>
- **Mathlib maintainers:**
<https://leanprover-community.github.io/teams/maintainers.html>
- **Mathlib reviewers:** <https://leanprover-community.github.io/teams/reviewers.html>
- **Review guidelines:**
<https://leanprover-community.github.io/contribute/pr-review.html>
- **Queueboard:** <https://leanprover-community.github.io/queueboard/>

Conclusions

Conclusions

- Anybody can contribute to mathlib and it's easier than it might seem!

Conclusions

- Anybody can contribute to mathlib and it's easier than it might seem!
- Don't be afraid to make mistakes!

Conclusions

- Anybody can contribute to mathlib and it's easier than it might seem!
- Don't be afraid to make mistakes!
- Your pull request will go through several review stages, don't be discouraged if it takes some time.

Conclusions

- Anybody can contribute to mathlib and it's easier than it might seem!
- Don't be afraid to make mistakes!
- Your pull request will go through several review stages, don't be discouraged if it takes some time.
- Mathlib is a huge project maintained by few (mostly as a side project) - help is always appreciated!

Conclusions

- Anybody can contribute to mathlib and it's easier than it might seem!
- Don't be afraid to make mistakes!
- Your pull request will go through several review stages, don't be discouraged if it takes some time.
- Mathlib is a huge project maintained by few (mostly as a side project) - help is always appreciated!
- Anybody can help reviewing!

Conclusions

- Anybody can contribute to mathlib and it's easier than it might seem!
- Don't be afraid to make mistakes!
- Your pull request will go through several review stages, don't be discouraged if it takes some time.
- Mathlib is a huge project maintained by few (mostly as a side project) - help is always appreciated!
- Anybody can help reviewing!

Thank you to Bryan Chen for the pretty statistics & to Max Kessler for the LaTeX template.

Conclusions

- Anybody can contribute to mathlib and it's easier than it might seem!
- Don't be afraid to make mistakes!
- Your pull request will go through several review stages, don't be discouraged if it takes some time.
- Mathlib is a huge project maintained by few (mostly as a side project) - help is always appreciated!
- Anybody can help reviewing!

Thank you to Bryan Chen for the pretty statistics & to Max Kessler for the LaTeX template.
Thank you for your attention. Feel free to ask any more questions.